Mazes vs Pathfinding (algorithms)

Robert A Lane

***Abstract*—READ CONC AFTER ABSTRACT TO DOUBLE CHECK IF PAPER IS WORTH USING**

This paper demonstrates my experiment to run various paathfinding algorithms against differeing maze generation algorithms.

I. Introduction

This investigation is a 'fun' visual way to compare search algorithms in their speed accuracy when navigating a variety of mazes generated by different algorithms. This should show some of the strengths and weaknesses of both search and maze algorithms. These maze algorithms are essential creating a spanning tree from the grid. As such we can not only use algrithms that generate trees e.g Binary Tree, but also ones that create minimum spanning trees within the graph.

II. Literature Review

might not need

III. Related work

what was the paper,defined by the author(surname) -¿ summarise the paper, bit you need?

[4] Vijaya et al discussed the use if mazes within game design. going into how different algorithms can lead to a sense of predictablility and therefore boredom when playing. this particular paper explores depth first search, in both solving the maze as well as generating one - both in the form of recursive backtracking algorithms.

[3] Narendran et al explore the effectiveness of various path finding algorithms within a maze. The authors compare fairly basic Breadth-first search, as well as depth-first search with A* search 2 variations aof Markov Decision Process as well as a Enhanced wall follower. The paper goes into depth of how many steps taken, memory usage and time complexity of these algorithhms. This study is particularly relevant as it is very similar to mine. I am differentiating through comparing the first 3 algorithms and last within mazes generated by different algorithms, as well as this these mazes, unlike this work, will not contain loops.

[2] James Buck Goes into varying detail about 15 different algorithms to generate mazes. ranging from the fairly standard recursive backtracker, to the wildly inefficient Aldous-Broder algorithm. This blog and his subsequent book [1] which goes into more detail about how to implement said algorithms and how they work, whilst also discussing how to add braiding, weaves and different grids.

[<**empty citation**>] research 4

[<**empty citation**>] research 5

[<**empty citation**>] research 6

i think mine is a planning around ai as seraching?? ai arent learning anything

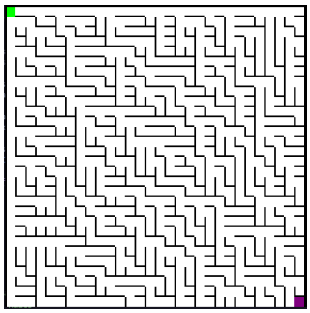

Fig. 1. generation of a binary tree maze given a random seed with a SE bias

IV. Implementation

using (git repos autho) repo (reference) we expanded on this to allow for multiple maze algorithms and pathfinders.

V. The Experiment

can make a perfect maze without any storage annndd if can garantee the choices(e.g with a seed?) don't even need to store whole maze. Due to the original code the gitRepo I forked off of (reference) there are limitations to the algorithms I have chosen. As such

VI. The Maze Algorithms

maze algorithms used will be from Jamis Buck's book Mazes for Programmers[1]

section on the maze algorithms/tree/graph gen algorithms I am using will have subsections

A. *binaryTree*

Algorithm 1 Binary Tree

boop

A very simple method of generating a maze is by utilising a binary tree. These mazes, however, are extremely biased. A binary tree maze can only pick 2 directions to move to. This creates a visual effect of a diagonal biased depending on the directions you choose the genral upside to this algorithm is that its firstly incredibly simple to implement -¿ show algorithm ¡-reference aswell as due to the way it generates can potentially create an infinite maze whilst only storing each cell.

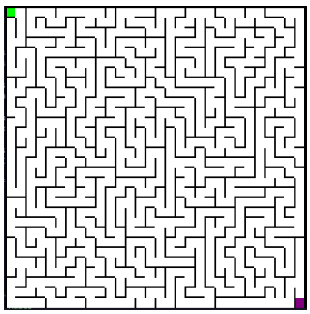

Fig. 2. Generation of recursive backtracker maze given a random seed

B. depth first/recursive backtracker

Algorithm 2 Recursive backtracker

boop

this is a recursive backtracker. The algorithm operates similarly to a depth first search: carving down a branch of a tree all the way till it ends, before looking at the other branches. It then backtracks to the first cell with unvisited neightbours and repeats until there are no unvisited cells. As figure 2 shows, this results in little to no bias in the generation as appose to a binary tree, see figure 1.

VII. The Pathfinding Algorithms

A. Brepth First

B. depthFirst

This is a greedy first algorithm which is essentially the same as the Recursive Backtracker :) only difference instead of creating the maze its solving it. This algorithm can be Modified with a heuristic as so the path picking isn't entirely random.

C. A*

D. wall Follower

section on pathfind/tree/graph search algorithms i am using USE ϵ-GREEDY-¿ with binaryTree due to the bias, compare 'with grain vs against grain' will have subsections

VIII. Results

IX. Conclusion

X. Future Work

References

[1] Jamis Buck. *Mazes for Programmers: Code Your Own Twisty Little Passages*. 1st edition. The Pragmatic Programmers. Dallas, Texas ; The Pragmatic Bookshelf, 2015. ISBN: 978-1-68050-131-5.

[2] *Maze Algorithms*. https://www.jamisbuck.org/mazes/. (Visited on 02/06/2026).

[3] Advik Narendran et al. "MazeSolver: Exploring Algorithmic Solutions for Maze Navigation". In: *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. June 2024, pp. 1–8. DOI: 10.1109/ICCCNT61001.2024.10725996. (Visited on 05/28/2026).

[4] J. Vijaya et al. "Analysis of Maze Generation Algorithms". In: *2024 5th International Conference on Innovative Trends in Information Technology (ICITIIT)*. Mar. 2024, pp. 1–6. DOI: 10.1109/ICITIIT61487.2024.10580178. (Visited on 02/16/2026).